

21-AI-PROMPT-LIBRARY.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-PROMPT-LIBRARY.md contient les modèles de prompts officiels utilisés pour guider les IA dans la construction, l'amélioration et la maintenance de Vitala.

Ces prompts doivent respecter :

- AI-CONSTITUTION.md ;
 - AI-PROMPT-STANDARD.md ;
 - AI-ARCHITECTURE-RULES.md.
-

2. Règles d'utilisation

Avant d'utiliser un prompt :

L'IA ou l'utilisateur doit :

- identifier le type de tâche ;
 - choisir le modèle adapté ;
 - compléter les informations manquantes ;
 - respecter les documents de référence.
-

3. Prompt général de développement

Usage

Créer une nouvelle fonctionnalité.

Tu travailles sur le projet Vitala.

Avant toute modification :

1. Analyse l'architecture existante.
2. Consulte les documents de référence.
3. Identifie les fichiers concernés.
4. Propose un plan.

Objectif :

[Décrire la fonctionnalité]

Contraintes :

- respecter l'architecture modulaire ;
- ne pas supprimer l'existant ;
- utiliser les conventions Vitala ;
- ajouter les tests nécessaires.

Livraison attendue :

- code complet ;
- fichiers modifiés ;
- tests ;
- documentation.

4. Prompt correction de bug

Usage

Corriger un problème existant.

Tu intervien sur Vitala pour corriger un bug.

Bug :

[Description]

Avant de corriger :

- analyse la cause réelle ;
- vérifie les impacts ;
- identifie les fichiers concernés.

Ne fais pas :

- correction temporaire ;

- contournement sécurité ;
- modification inutile.

Livraison :

- cause du problème ;
- correction appliquée ;
- tests effectués ;
- risques éventuels.

5. Prompt audit architecture

Usage

Vérifier la qualité d'un module.

Réalise un audit architectural du module :

[Nom du module]

Vérifie :

- séparation des responsabilités ;
- organisation des fichiers ;
- dépendances ;
- sécurité ;
- maintenabilité ;
- performances.

Produit :

1. Problèmes trouvés.
2. Niveau de gravité.
3. Recommandations.
4. Plan d'amélioration.

6. Prompt création d'un module complet

Usage

Créer un nouveau domaine Vitala.

Crée le module :

[Nom du module]

Respecte :

- architecture par domaine ;
- structure officielle feature ;
- types séparés ;
- services séparés ;
- tests ;
- documentation.

Le module doit être :

- autonome ;
- sécurisé ;
- scalable ;
- maintenable.

Avant de coder :

présente l'architecture proposée.

7. Prompt Supabase / Database

Usage

Modifier la base de données.

Tu travailles sur la couche Database Vitala.

Objectif :

[Modification demandée]

Respecte :

- PostgreSQL ;
- migrations versionnées ;
- DATABASE-Dictionary.md ;
- DATABASE-RLS-MATRIX.md.

Avant modification :

analyse :

- tables concernées ;
- relations ;
- sécurité ;
- impact API.

Livraison :

- migration ;
- changements RLS ;
- tests ;
- documentation.

8. Prompt API

Usage

Créer ou modifier une API.

Travaille sur l'API Vitala.

Objectif :

[Endpoint ou fonctionnalité]

Respecte :

- conventions REST ;
- versionnement ;
- sécurité ;
- gestion des erreurs.

Fournis :

- endpoint ;
 - méthode ;
 - paramètres ;
 - réponses ;
 - erreurs possibles ;
 - tests.
-

9. Prompt Frontend

Usage

Créer une interface.

Crée ou améliore l'interface :

[Nom]

Respecte :

- Design System Vitala ;
- composants existants ;
- accessibilité ;
- responsive ;
- performance.

Ne crée pas :

- composants dupliqués ;
- logique métier dans l'UI.

Livraison :

- composants ;
 - hooks ;
 - services nécessaires ;
 - tests.
-

10. Prompt sécurité

Usage

Audit sécurité.

Réalise un audit sécurité de :

[Module]

Analyse :

- authentification ;

- permissions ;
- données sensibles ;
- API ;
- Database ;
- RLS ;
- secrets.

Produit :

- vulnérabilités ;
- gravité ;
- corrections recommandées.

11. Prompt refactoring

Usage

Améliorer du code existant.

Analyse ce code avant refactoring.

Objectif :

[Amélioration]

Ne modifie pas le comportement existant.

Vérifie :

- risques ;
- dépendances ;
- tests existants.

Propose :

1. Problème actuel.
 2. Solution.
 3. Plan progressif.
 4. Validation.
-

12. Prompt optimisation performance

Usage

Améliorer la vitesse.

Analyse les performances de :

[Module]

Cherche :

- lenteurs ;
- requêtes inutiles ;
- rendu excessif ;
- consommation mémoire.

Propose des améliorations mesurables.

Ne sacrifie jamais :

- sécurité ;
 - lisibilité ;
 - architecture.
-

13. Prompt préparation production

Usage

Valider une release.

Prépare cette fonctionnalité pour production.

Vérifie :

- tests ;
- sécurité ;
- migrations ;
- monitoring ;
- documentation ;
- rollback.

Retour attendu :

Checklist complète de production.

14. Prompt analyse d'un projet existant Lovable

Usage

Reprendre un projet généré automatiquement.

Analyse le projet existant.

Objectif :

Transformer le projet vers l'architecture Vitala.

Analyse :

- structure actuelle ;
- fichiers inutiles ;
- dette technique ;
- sécurité ;
- modularité.

Ne supprime rien sans justification.

Produit :

- état actuel ;
- problèmes ;
- plan de reconstruction progressive.

15. Prompt multi-IA

Usage

Faire travailler plusieurs IA.

Tu es une IA spécialisée dans un domaine précis de Vitala.

Respecte :

- AI-CONSTITUTION ;
- architecture officielle ;
- documentation existante.

Ne modifie pas les domaines hors périmètre.

Documente chaque décision importante.

16. Checklist utilisation d'un prompt

☐ Type de tâche identifié

☐ Documents consultés

☐ Objectif clair

☐ Contraintes définies

☐ Validation définie

☐ Résultat attendu précisé

17. Règle finale

Un bon prompt Vitala ne donne pas seulement une instruction.

Il transmet une responsabilité complète à l'IA.

Historique

Version	Modification
V1.0	Création bibliothèque prompts IA

Fin de AI/21-AI-PROMPT-LIBRARY.md